RFC: A Simple VFD Configuration Language

John Mainzer john.mainzer@lifeboat.llc

As discussed in "RFC: A Plugin Interface for HDF5 Virtual File Drivers" [1], a text based configuration language for VFDs would allow VFD agnostic configuration of VFD stacks via either environment variable or command line parameter. This RFC is a simplified rewrite of Appendix A, combined with an outline of the current parser implementation and initial integration into the HDF5 library.

1 Introduction 2
2 A Grammar for VFD Stack Configuration 3
3 Parser Implementation 5
4 Integration into HDF5 11
5 Integration into VFDs 12
6 Testing 13

© 2026 Lifeboat, LLC

1 Introduction

At present, VFDs are stacked in the HDF5 library by constructing "stacks" of FAPLs, to setup the required VFD stack on file open. Here, each entry in the stack contains the driver information message required to configure the VFD at the associated level in the VFD stack.

To make this concept more concrete, consider the following code to construct the stack of FAPLs needed to setup native encryption in the HDF5 library.

In this example, the top level VFD is a page buffer VFD used to convert random I/O to paged I/O. The encrypting VFD is at the next level – it encrypts and decrypts pages as required. Finally, the sec2 VFD appears at the bottom of the stack to perform actual disk I/O.

```
hid_t pb_fapl_id        = H5I_INVALID_HID;
hid_t crypt_fapl_id     = H5I_INVALID_HID;
hid_t sec2_fapl_id      = H5I_INVALID_HID;
H5FD_pb_vfd_config_t pb_vfd_config =
{
    /* magic          = */ H5FD_PB_CONFIG_MAGIC,
    /* version        = */ H5FD_CURR_PB_VFD_CONFIG_VERSION,
    /* page_size      = */ 4096,
    /* max_num_pages  = */ 64,
    /* rp             = */ H5FD_PB_DEFAULT_REPLACEMENT_POLICY,
    /* fapl_id        = */ H5P_DEFAULT, /* will overwrite */
    /* testing        = */ H5FD_PB_DEFAULT_TESTING_OFF
};
H5FD_crypt_vfd_config_t  crypt_vfd_config =
{
    /* magic                  = */ H5FD_CRYPT_CONFIG_MAGIC,
    /* version                = */ H5FD_CURR_CRYPT_VFD_CONFIG_VERSION,
    /* plaintext_page_size    = */ 4096,
    /* ciphertext_page_size   = */ 4112,
    /* encryption_buffer_size = */
                                   H5FD_CRYPT_DEFAULT_ENCRYPTION_BUFFER_SIZE,
    /* cipher                 = */ 0,
    /* cipher_block_size      = */ 16,
    /* key_size               = */ 32,
    /* key                    = */ H5FD_CRYPT_TEST_KEY,
    /* iv_size                = */ 16,
    /* mode                   = */ 0,
    /* fapl_id                = */ H5P_DEFAULT  /* will overwrite */
};
/* first setup the sec2 fapl id */
sec2_fapl_id = H5Pcreate(H5P_FILE_ACCESS);
H5Pset_fapl_sec2(sec2_fapl_id);
/* next setup the encryption fapl id */
crypt_fapl_id = H5Pcreate(H5P_FILE_ACCESS);
crypt_vfd_config.fapl_id = sec2_fapl_id;  /* setup underlying fapl id */
H5Pset_fapl_crypt(crypt_fapl_id, &crypt_vfd_config);
/* finally setup the top level fapl that will be used in the call to
 * H5Fopen()
 */
```

```
pb_fapl_id = H5Pcreate(H5P_FILE_ACCESS);
pb_vfd_config.fapl_id = crypt_fapl_id; /* setup the underlying fapl ID */
H5Pset_fapl_pb(pb_fapl_id, &pb_vfd_config);
```

The above code demonstrates the construction of this stack of FAPLs. In this example, the `pb_fapl_id` would be passed in the H5Fopen call to construct the indicated VFD stack. While this approach is awkward, it works well enough for many purposes.

What it doesn't do well is allow us to pass VFD stack specifications to tools (e.g.,h5dump), particularly if some or all of the members of the VFD stack must be loaded dynamically.

As discussed in "RFC: A Plugin Interface for HDF5 Virtual File Drivers" [1], the notion of a text based language for configuring VFD stacks has been around for some time. This document presents a simplified version of the grammar presented in Appendix A of [1], discusses the requirements and implementation details of the associated parser, and outlines the limited integration of this configuration language into the HDF5 library and into the prototype page buffer and encryption VFDs.

2 A Grammar for VFD Stack Configuration

The purpose of a grammar for VFD stack configuration is to facilitate reasonably efficient expression of VFD stacks. This requires balancing the following grammar design objectives:

1. Grammar must facilitate breadth first parsing
2. Parser must be easy to implement and maintain
3. Grammar must be flexible enough to express all needed VFD configuration data
4. VFD configurations strings in the language should be readable and reasonably compact
5. Grammar and implementation must be extendible if new requirements emerge in the future

While most of these objectives are obvious, the requirement that the grammar facilitate breadth first parsing may not be. To see this, consider that each VFD on the stack will not in general know anything about the configuration data for the underlying VFDs. Thus, the grammar must facilitate parsing configuration data for the current VFD, while allowing easy recognition of sections of the configuration string pertaining to the underlying VFD(s) so that those sub-string(s) can be passed down the stack to allow configuration of the underlying VFD(s).

This issue can be addressed with breadth first parsing. Observe that this is a departure from typical parsing techniques – outside of natural language and AI applications, the use of depth first parsing is near universal.

As shall be seen, a grammar that uses matching parentheses around each statement facilitates breadth first parsing nicely, albeit at the cost of some extra syntactic fluff.

Due to the breadth first parsing requirement, no real attempt has been made to find an existing grammar and parser to adapt to this purpose. Given the simplicity of the grammar and the near universal use of depth first parsing, chances of finding anything useful seemed vanishingly small.

With these points addressed, consider the following grammar:

© 2026 Lifeboat, LLC

```
<name_value_pair> ::= '(' <identifier> <value> ')'

<value> ::= <integer> | <float> | <quote_string> | <binary_blob> |
<name_value_pair_list>

<name_value_pair_list> ::= '(' (<name_value_pair>)* ')'
```

where the non-terminals not defined above are loosely defined below:

`<identifier>`	a valid C identifier
`<integer>`	a C integer constant
`<float>`	a C floating point constant
`<quote_string>`	a C quote string
`<binary_blob>`	a hex representation of an arbitrary sequence of binary bytes with a "--" prefix to distinguish it from identifiers and integer constants[1]

As the astute reader will note, this is just a restricted definition of a LISP form.

To turn this into a workable configuration language, some semantic constraints are required.

In the context of a VFD configuration, the top level must be a single `<name_value_pair>`, where the identifier is the name of the VFD, and the value must be a (possibly empty) `<name_value_pair_list>`[2]. This list of name value pairs must contain the configuration data required by the target VFD, with the list of identifiers that may appear, and the range and type of each associated value specified by the target VFD. As a result, the maximum length of this list of name value pairs is also specified by the target VFD.

Leaving aside the matter of underlying VFDs, the values associated with each of these name-value pairs will be either integer, float, quote string, or binary blob. While this selection of types is sufficient for all VFDs that we are aware of, new types can be added as necessary.

[1] There is nothing magic about the choice of the "--" prefix. Any prefix that can't appear at the beginning of either an integer, float, or quote string will be fine. To allow for the possibility of enumerated values in the future, the prefix should also not appear at the beginning of identifiers.

[2] This is slightly different from the grammar and examples shown in appendices A and B of "RFC: A Plugin Interface for HDF5 Virtual File Drivers". The list of name value pairs is enclosed in parentheses to simplify the grammar and facilitate breadth first parsing.

© 2026 Lifeboat, LLC

For underlying VFDs, the value must be a single name value pair, where the name is the name of the underlying VFD, and the value is another (possibly empty) list of name value pairs containing the necessary configuration information.[3]

While it should be obvious that this format allows definition of an arbitrary, possibly tree structured, VFD stacks, how strings in this language are parsed and used to setup the desired VFD stack is discussed in subsequent sections.

3 Parser Implementation

Parsing techniques are text book information. Thus, this section presumes a nodding familiarity with the subject and concentrates on deviations from typical parser design.

Observe that the above grammar is easily parsed with a recursive descent parser.

Recursive descent parsers have largely gone out of style today, as there are many parser generators. However, they are easy to write and maintain for simple grammars such as the one presented above. Further, since the parser is coded directly, it is easy to pass a flag to the `lexer` telling it when the value of a name value pair is expected.

This allows the `lexer` to be context sensitive. If value expected flag is set, and the next non-blank character in the input stream is a left parentheses, the `lexer` treats the input sub-string from the left parenthesis up through the matching right parenthesis as a single token. Without this flag, the `lexer` would recognize the left parenthesis as token in its own right.

This context sensitivity allows the desired breath first parse, since the configuration string for any underlying VFD stack is "lexed" as a single token.

The recursive descent parser for the above grammar consists of two functions – one to parse a `<name_value_pair>` and load its name and value into an instance of H5CL_nv_pair_t, and one to parse a `<name_value_pair_list>`, and load the names and values of its contents into an array of `H5CL_nv_pair_t`. These functions are implemented in H5CL.c and have the following signatures:

```
H5_DLL herr_t
H5CL__parse_name_value_pair(H5CL_nv_pair_t *nv_pair_ptr,
                            H5CL_lex_vars_t * lex_vars_ptr);
H5_DLL herr_t
H5CL__parse_name_value_pair_list(H5CL_nv_pair_t * nv_pairs,
                                 int max_nv_pairs,
                                 H5CL_lex_vars_t * lex_vars_ptr);
```

where:

`nv_pair_ptr` is a pointer to a single instance of `H5CL_nv_pair_t`,

`nv_pairs[]` is an array of `H5CL_nv_pair_t` of length `max_nv_pairs`, and

[3] While not useful for configuring VFD stacks from configuration strings stored in a configuration file or environment variable, in principle, the grammar allows the underlying VFD to be specified as an integer – specifically the ID of the fapl containing the configuration data for the underlying VFD.

`lex_vars_ptr` is a pointer to an instance of `H5CL_lex_vars_t` containing the configuration string and the state of the `lexer`.

Both functions return `SUCCEED` on success, and `FAIL` if any error is detected. In particular, `parse_name_value_pair_list()` must fail if more than `max_nv_pairs` name value pairs are encountered. It is the responsibility of the caller to generate an error if required name value pairs are omitted, unknown name value pairs appear, or if the supplied values are in any way invalid.

As mentioned above, the name and value of each successfully parsed name value pair is stored in an instance of `H5CL_nv_pair_t`, and contents of a name value pair list is stored in an array of `H5CL_nv_pair_t`.

Conceptually, when a VFD that supports configuration via the configuration language detects a configuration string, it allocates an array of `H5CL_nv_pair_t` of appropriate size, and passes the configuration string and this array to the parser. If the parser is successful, on return the array of `H5CL_nv_pair_t` will be loaded with the names and values of the name value pairs directed at the VFD in question. The VFD in question then scans this array to extract the configuration data.

`H5CL_nv_pair_t` is defined in `H5CLdevelop.h`, and is reproduced below:

```
/****************************************************************************
*
* struct H5CL_nv_pair_t
*
* The structure used to store the name and value from a successfully parsed
* name value pair.
*
* The fields in the structure are discussed individually below.
*
* struct_tag:  unsigned integer which must always contain the the value
*              H5CL_NV_PAIR_STRUCT_TAG.  The struct_tag field allows us to
*              verify that a pointer to struct H5CL_nv_pair_t does in
*              fact point to an instance of same.
*
* name_ptr:    Pointer to a dynamically allocated string containing the
*              name in the name value pair, or NULL if the name is
*              undefined.
*
* val_type:    Integer code indicating the type of the value stored in
*              this instance of H5CL_nv_pair_t.  Possible values are:
*
*              H5CL_VAL_NONE: The name value pair is undefined.
*
*              H5CL_VAL_INT: The value associated with the name value
*                      pair is an integer stored in int_val field.
*
*              H5CL_VAL_FLOAT: The value associated with the name value
*                      pair is an integer stored in f_val field.
*
*              H5CL_VAL_QSTR: The value associated with the name value
*                      pair is a quote string stored without its leading
*                      and trailing double quotes and with a terminating
*                      null char (\0) in a dynamically allocated vector
*                      of char pointed to by the vlen_val field.  The
*                      length of this string is stored in the len field.
*                      Note that this len does not include the terminating
```

```
*                       null char.
*
*               H5CL_VAL_BB: The value associated with the name value
*                       pair is a binary blob stored in a dynamically
*                       allocated vector of uint8_t pointed to by the
*                       vlen_val field  The length of this vector is stored
*                       in the len field.
*
*               H5CL_VAL_LIST: The value associated with the name value
*                       pair is a configuration language sub expression
*                       most likely containing configuration data for
*                       underlying VFD(s).  It is stored as a string in
*                       a dynamically allocated vector of char.  The
*                       length of the sub expression (less the terminating
*                       null char) is stored in the len field.
*
* int_val:      int64_t containing the integer value associated with the
*               name value pair if val_type == H5CL_VAL_INT.  In all other
*               cases, int_val should be set to 0.
*
* f_val:        double containing the floating point value associated with
*               the name value pair if val_type == H5CL_VAL_FLOAT.  In all
*               other cases, f_val should be set to 0.0.
*
* vlen_val_ptr: void pointer whose value depends on the value of val_type.
*
*               If val_type == H5CL_VAL_QSTR, vlen_val_ptr points to a
*               dynamically allocated vector of char containing a text
*               string of length len.  In this case, the string contains a
*               quote string less its leading and trailing double quotes.
*               Note that any escape sequences in the string have not been
*               resolved by the lexer.
*
*               If val_type == H5CL_VAL_BB, vlen_val_ptr points to a
*               dynamically allocated vector of uint8_t that contains a
*               binary blob of length len.
*
*               If val_type == H5CL_VAL_LIST, vlen_val_ptr points to a
*               dynamically allocated vector of char containing a sub-
*               expression in the configuration language expressed as a null
*               terminated C string.  This sub-expression will typically
*               contain configuration data for underlying VFD(s).  The length
*               of this string (less the terminating null char) is stored in
*               len.
*
*               In all other case, vlen_val_ptr should be set to NULL.
*
* len           size_t field containing the length of the string or
*               binary blob pointed to by vlen_val, or 0 if vlen_val is NULL.
*
****************************************************************************/

#define H5CL_NV_PAIR_STRUCT_TAG           0x007A
#define H5CL_INVALID_NV_PAIR_STRUCT_TAG   0x07A0

typedef struct H5CL_nv_pair_t
{
    uint32_t struct_tag;
```

```
    char *name_ptr;
    int val_type;
    int64_t int_val;
    double f_val;
    void *vlen_val_ptr;
    size_t len;

} H5CL_nv_pair_t;
```

In addition to its context sensitivity and return of configuration data in an array of `H5CL_nv_pair_t`, the parser has a number of other unusual features to address the peculiarities of the VFD configuration problem.

Of the remainder, the most obvious of these is the relatively small size of configuration strings. In particular, it is difficult to imagine a VFD stack configuration string larger than five or so kilobytes. Further, configuration strings will typically be communicated via either environment variables or command line options. Thus, instead of reading the configuration from a file line by line as is typical, the entire configuration string is loaded at once, and then parsed.

Further, the breadth first parse requires that each time we move down the stack, we set up a new input string, containing only that portion of the original configuration string that pertains to the underlying VFD stack.

Because of these points, it is convenient to set up a structure encapsulating the input string, the current state of the `lexer`, and the current token. This is the `H5CL_lex_vars_t` structure mentioned above and defined in `H5CLpkg.h`. Its definition and that of the associated `H5CL_token_t` structure are reproduced below:

```
/*****************************************************************************/
/* The H5FDL_TOKEN #defines are used to indicate the type of a token      */
/* recognized by the lexical analyzer.                                     */
/*****************************************************************************/

#define H5CL_ERROR_TOK                      0
#define H5CL_L_PAREN_TOK                    1
#define H5CL_R_PAREN_TOK                    2
#define H5CL_SYMBOL_TOK                     3
#define H5CL_INT_TOK                        4
#define H5CL_FLOAT_TOK                      5
#define H5CL_QSTRING_TOK                    6
#define H5CL_BIN_BLOB_TOK                   7
#define H5CL_LIST_TOK                       8
#define H5CL_EOS_TOK                        9

/*****************************************************************************
 *
 * struct H5CL_token_t
 *
 * The token structure is used to store tokens as they are read from the
 * input string by the lexical analyzer.  The fields in the structure are
 * discussed individually below.
 *
 * struct_tag:  unsigned integer which must always contain the the value
 *              H5CL_TOKEN_STRUCT_TAG.  The struct_tag field allows us to
 *              verify that a pointer to struct H5CL_token_t does in fact
```

```
 *              point to an instance of same.
 *
 * code:        Integer code indicating the type of the token.  The set of
 *              allowable values for this field is equal to the set of
 *              H5CL_*_TOK #defines earlier in this file.
 *
 * str_ptr:     Pointer to char.  str_ptr points to a dynamically allocated
 *              string which contains a text representation of the token.
 *              Note that in the case of numerical values, *str_ptr need not
 *              agree with val, as *str_ptr may contain a text
 *              representation of an out of range value.
 *
 *              The buffer pointed to by str_ptr is recycled, and at
 *              present, will always be of the same length as the input
 *              string in the containing instance of H5CL_lex_vars_t.
 *
 * str_len:     size_t containing the length of the null terminated string
 *              pointed to by str_ptr.
 *
 * max_str_len: size_t containing the number of bytes in the buffer pointed
 *              to by str_ptr.  Note that should always be larger than
 *              str_len.  At present, max_str_len will always be the same as
 *              the length of the input string in the containing instance of
 *              of H5CL_lex_vars_t
 *
 * int_val:     int64_t containing any integer value associated with the
 *              instance of H5CL_token_t.
 *
 * f_val:       Double containing any floating point value associated with
 *              the instance of H5CL_token_t.
 *
 * bb_ptr:      Pointer to a dynamically allocated vector of uint8_t of
 *              length equal to max_str_len.  This vector is used to store
 *              the value of a binary blob. Note that the size of the buffer
 *              pointed to bb_ptr is always max_str_len.
 *
 * bb_len       size_t field containing the number of bytes in the binary
 *              blob. As per str_len, this value must always be less than
 *              max_str_len.
 *
 ****************************************************************************/

#define H5CL_TOKEN_STRUCT_TAG           0x005A
#define H5CL_INVALID_TOKEN_STRUCT_TAG   0x05A0

typedef struct H5CL_token_t
{
       uint32_t struct_tag;
       int32_t code;
       char *str_ptr;
       size_t str_len;
       size_t max_str_len;
       int64_t int_val;
       double f_val;
       uint8_t * bb_ptr;
       size_t bb_len;

} H5CL_token_t;
```

```
/****************************************************************************
 *
 * struct H5CL_lex_vars_t
 *
 * A single instance of H5CL_lex_vars_t is used to consolidate all the
 * variables employed by the configuration language lexer code. The fields
 * of this structure are discussed individually below.
 *
 * struct_tag:  Unsigned integer which must always contain the the value
 *              H5CL_LEX_VARS_SRUCT_TAG.  The struct_tag field allows us to
 *              easily verify that a pointer to struct H5CL_lex_vars_t does
 *              in fact point to an instance of same.
 *
 * input_str_ptr: Pointer to a dynamically allocated string that has been
 *              loaded with the configuration language string to be lexed.
 *
 * next_char_ptr: Pointer to the next character to be lexed.  This pointer
 *              is initialized to input_str_ptr, and then incremented as
 *              tokens are recognized.
 *
 * end_of_input: Boolean that is initialized to false, and set to true when
 *              the end of the input string is reached.
 *
 * err_ctx:     Array of char of length H5CL_ERR_CTX_LEN + 1 used to stage
 *              a string containing a substring of the input string
 *              centered at the point at which a syntax error was detected.
 *              This string is used to provide context for error messages.
 *
 * token:       Instance of struct H5CL_token_t.  A pointer to this instance
 *              is returned by the lexer, and reused for each new token
 *              recognized.
 *
 ****************************************************************************/

#define H5CL_LEX_VARS_STRUCT_TAG                0x006A
#define H5CL_INVALID_LEX_VARS_STRUCT_TAG        0x06A0
#define H5CL_ERR_CTX_LEN                        30
#define H5CL_MAX_ERR_MSG_LEN                    (128 + H5CL_ERR_CTX_LEN + 6)

typedef struct H5CL_lex_vars_t
{
   uint32_t struct_tag;
   char * input_str_ptr;
   char * next_char_ptr;
   bool end_of_input;
   char err_ctx[H5CL_ERR_CTX_LEN + 7];
   struct H5CL_token_t token;

} H5CL_lex_vars_t;
```

The functions `H5CL__init_lex_vars()` and `H5CL__take_down_lex_vars()` are used to setup and take down this structure. These functions are defined in `H5CL.c` and their signatures are reproduced below:

```
H5_DLL herr_t
```

```
H5CL__init_lex_vars(const char * input_str_ptr,
                    H5CL_lex_vars_t * lex_vars_ptr);

H5_DLL herr_t
H5CL__take_down_lex_vars(H5CL_lex_vars_t * lex_vars_ptr);
```

Finally, note that the `lexer` uses the C library routines `strtoll()` and `strtod()` to translate ASCII representations of integers and floats to their binary equivalents.

4 Integration into HDF5

Thanks to Jordan's work on support for pluggable VFDs, the configuration language can be integrated into the HDF5 library with minimal changes. This works because of the `H5FD_driver_prop_t` structure defined in `H5FDprivate.h` and reproduced below:

```
/* Define structure to hold driver ID, info & configuration string for FAPLs
*/
typedef struct {
   hid_t       driver_id;         /* Driver's ID */
   const void *driver_info;       /* Driver info, for open callbacks */
   const char *driver_config_str; /* Driver configuration string */
} H5FD_driver_prop_t;
```

Instances of `H5FD_driver_prop_t` are used to store VFD configuration information in the file access property list with the `fapl_id` identifier, with the `driver_id` field used to store the `ID` of the desired VFD. If provided, configuration data is either in a structure pointed to by the `driver_info` field, or in a string pointed to by the `driver_config_str` field.

With this infrastructure in hand, we can repeat our earlier example loading VFD stack configuration data into `fapl_id` as follows:

```
char vfd_stack_config_str[] =
"( page_buffer "
" (( page_size 4096 )"
"  ( max_num_pages 64 )"
"  ( replacement_policy 0 )"
"  ( underlying_VFD "
"   ( encryption_VFD "
"    (( plaintext_page_size 4096 )"
"     ( ciphertext_page_size 4112 )"
"     ( encryption_buffer_size 65792 )"
"     ( cipher 0 )"
"     ( cipher_block_size 16 )"
"     ( key_size 32 )"
"     ( key --0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF )"
"     ( iv_size 16 )"
"     ( mode 0 )"
"     ( underlying_VFD ( sec2 () ) )"
"    )"
"   )"
"  )"
" )"
")";
hid_t fapl_id = H5I_INVALID_HID;
```

```
fapl_id = H5Pcreate(H5P_FILE_ACCESS);

H5CL_load_vfd_config_str_into_fapl(fapl_id, vfd_stack_config_str);
```

As with the prior example, `fapl_id` would be passed in a subsequent `H5Fopen` call to setup the desired VFD stack.

In the above example, `H5CL_load_vfd_str_into_fapl()` is a helper function currently defined in `H5CL.c`. Its signature is as follows:

```
H5_DLL herr_t
H5CL_load_vfd_config_str_into_fapl(hid_t fapl_id, char *vfd_config_str_ptr);
```

Briefly, this function parses the top-level name value pair in `*vfd_config_str_ptr` to obtain the name of indicated VFD, looks up its `ID`, and then calls `H5P_set_driver()`, to load these values into `fapl_id` – the file access property list for the file driver.

Obviously, a public API version of this function will be needed eventually.

Before proceeding to a discussion of necessary VFD modifications, it may be useful to mention the existence of the `H5P__facc_set_def_driver()` function in `H5Pfapl.c`.

To simplify greatly, this function examines the environment variables "`HDF5_DRIVER`" and "`HDF5_DRIVER_CONFIG`" looking for a VFD name and associated configuration string. If successful, it uses this information to set the file driver file access property list `fapl_id` – thus setting the default VFD stack.

While it has not been tested, this suggests that with the addition of the configuration language code and the addition of support for configuration language strings in the desired VFDs, we have everything we need to configure tools to setup VFD stacks as directed by environment variables.

5 Integration into VFDs

In addition to integration into the HDF5 library proper, it is also necessary to integrate configuration language support into the target VFDs. This has been done for the initial implementation of page buffer and encryption VFDs. It is not necessary for the `SEC2` VFD, as that VFD does not require configuration.

Sadly, VFDs are far from being standardized, so this process will be different for different VFDs. However, a conceptual outline of how this has been done to date may be useful.

With the exception of some tidy up in the close routine, so far it has been sufficient to modify the configuration load code in the open callbacks to load configuration data as follows:

- Peek at the instance of `H5FD_driver_prop_t` in the supplied `fapl` and make note of the contents of the `driver_info` and `driver_config_str` fields.
- If the `driver_info` field is not `NULL`, coerce its void pointer to a pointer to the binary config structure associated with the VFD, load this configuration data as usual, and then proceed with setting up the VFD as before.
- If the `driver_config_str` pointer is not `NULL`:

- Recall that the VFD specific extension of `H5FD_t` typically contains an instance of the VFD specific configuration structure. Initialize all fields in this structure to their default values.
- Allocate an array of `H5CL_nv_pair_t` of length sufficient to contain all configuration data. Initialize the `struct_tag` field and call `H5CL_init_nv_pair()` on each entry in this array.
- Call `H5CL_parse_config()`, passing it the name of the VFD, the configuration string, and the base address and length of the array of `H5CL_nv_pair_t`.

  `H5CL_parse_config()` verifies that the top level name value pair has name matching the supplied VFD name, and then parses the associated name value pair list and loads the name value pairs into the array of `H5CL_nv_pair_t`.
- After `H5CL_parse_config()` returns, scan the array of `H5CL_nv_pair_t`, being sure to validate the values associated with each name value pair. Copy this data into the VFD specific binary configuration structure in the VFD specific extension of `H5FD_t`. Throw an error if any required values are missing, if any values are out of range, or if any unknown configuration parameter names appear.

  If the name value pair specifies the configuration string for an underlying VFD stack, some additional processing is required. A new `fapl` must be created, and loaded with the target VFD `ID` and configuration string via a call to `H5CL__load_vfd_config_str_into_fapl()`. The new `fapl` is loaded into the appropriate field in the binary configuration structure.
- This done, the necessary configuration data has been loaded into the VFD configuration structure in the VFDs extension of `H5FD_t`. At this point, the open can proceed as usual.

- If both the `driver_info` and the `driver_config_str` fields are `NULL`, load default values into the VFD specific configuration structure in its extension of `H5FD_t`, and proceed with the open as usual. Note that this is not always practical. For instance, it doesn't make sense to configure the encrypting VFD with a default key.

By using the configuration string to populate the binary configuration structure in the target VFDs extension of `H5FD_t`, we can avoid other changes to the VFD. In particular, there is no need to modify the `fapl_get` callback, as it continues to work with the binary representation of the configuration data as before.

6 Testing

The configuration language is implemented as the new `H5CL` package in the source directory, and tested in `vfd_cl.c` in the test directory. The existing test suite is relatively complete, and includes tests for both correctness and error detection and reporting via the HDF5 error stack.

In contrast, testing of configuration language integration into the page buffer and encryption VFDs simply re-runs the existing tests only with configuration strings instead of the pre-existing binary configuration. Since the existing tests are inadequate, so are the tests using configuration strings.

References

1. J. Smith, J. Henderson, "RFC: A Plugin Interface for HDF5 Virtual File Drivers", https://support.hdfgroup.org/releases/hdf5/documentation/rfc/RFC_VFD_Plugin.docx.pdf

Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0024823 for Phase I and Phase II SBIR project "Protecting the confidentiality and integrity of data stored in HDF5"

Revision History

October 17, 2024	Version 1 was created from original document from 9/19/2024.
October 12, 2025	Version 2 updated binary blob description and minor copy editing.
March 17, 2026	Version 3 updated to reflect initial implementation and integration into version 1.14.6 of the HDF5 library.
March 23, 2026	Version 4 updated to address comments.
March 24, 2026	Version 5 updated to address comments.

© 2026 Lifeboat, LLC